ESP32 C6 testbench wiring and voltage scaling guide

Prepared for Jonathan Quiles • 6 October 2026 • Revision 2 expanded technical guide

This guide documents the client simulator terminal assignments, measured signals, proposed ESP32-C6 connections, and the steps for bringing actual readings into the dashboard. The testbench is powered from 12 V; the ESP32 is powered by USB. No actual motorcycle ECU is connected. O2 acquisition has been tested successfully. The other analog channels require additional voltage scaling before GPIO connection.

Project overview

The immediate objective is to display real simulator outputs without exposing ESP32 inputs to excessive voltage. O2 follows its physical knob, with a reported maximum of 1.255 V on the ESP32. MAP, IAT, ECT and TPS need scaling. RPM and switching interfaces remain unverified.

Use this guide to build and verify four independent dividers, preserve raw measurement evidence, and identify the calibration needed for useful engineering readings. Sensor explanations and original client specifications are included alongside the wiring instructions. Live input monitoring does not establish that an ECU or ABS module works.

Contents

Project overview 1

1 Current verification status 3

2 Power and ground 3

3 Connection map 4

4 Voltage divider circuit 4

5 Parts and equipment 5

6 Assembly and commissioning procedure 6

7 Arduino voltage acquisition 6

8 Dashboard and calibration behavior 7

9 Measurement record 8

10 What this testbench is intended to do 8

11 Client requirements and differences from actual measurements 9

12 Engine sensor meanings and testing context 10

13 RPM wheel speed and switching signals 12

14 ABS and traction control 13

15 Other display fields and source identification 14

16 Firmware and dashboard versions 14

17 Calibration and acceptance evidence 15

18 Troubleshooting 16

19 Next implementation sequence 16

20 References 17

1 Current verification status

The following results were reported during testing. They are measurements of this simulator, not universal motorcycle sensor specifications.

Signal	Simulator terminal	Reported readings	Status
O2	6	Meter maximum 1.248 V; ESP32 maximum 1.255 V	Reading responds to the physical knob. Basic acquisition verified at the tested setting; full calibration not established.
MAP	7	Maximum 4.25 V	Needs scaling. Minimum not recorded.
IAT	4	1.660-3.385 V	Needs scaling. Knob temperature direction is separate from numeric voltage minimum/maximum.
ECT	5	High end 3.575 V; low end meter showed zero	Needs scaling. Low reading remains unverified.
TPS	8	High end 4.26 V; low end meter showed zero	Needs scaling. Low reading remains unverified.
RPM	3	Frequency signal confirmed by client	Pulse voltage, waveform and frequency-to-RPM calibration unknown.
Switches	9 and 10	Client says one is brake	Identities, active states and voltage levels unknown.

The HT123's stated DC voltage range starts at 0.8 V. Its zero readings at low settings do not establish a true zero output. Use a suitable low-voltage meter to verify those endpoints. Its AC frequency-response specification is not a frequency measurement feature.

The 7 mV difference between the O2 meter and ESP32 maxima is approximately 0.6%. This is agreement at one tested point, not proof of accuracy over the full range or proof that an ECU works.

2 Power and ground

Continue using the existing 12 V supply for the testbench and USB for the ESP32. Connect simulator signal ground to ESP32 GND only after confirming the correct ground terminal. Terminals 1 and 2 are labeled GND; confirm they are common. Never connect the testbench's 12 V supply to an ESP32 GPIO, 3V3 or 5V0 pin.

Leave terminal 11 (3.3 V) and terminal 12 (5 V) disconnected from the ESP32 during USB-powered testing. The client identifies them as ESP power supplies, but their regulation, load capacity and the clone development board's power routing have not been checked.

If standalone simulator power is used later, verify the regulated supply and board power circuit first. Use one power arrangement at a time unless the board's circuit explicitly supports simultaneous sources. The 5 V header pin is labeled 5V0 on the supplied board image; it is a power pin, not a signal pin.

3 Connection map

Use printed GPIO labels, not physical header positions. The ESP32-C6 has ADC channels on GPIO0-6. Our analog allocation uses GPIO0-4. GPIO4-7 also have external JTAG functions; this plan assumes external JTAG is not in use. Keep native USB GPIO12/13 available and do not substitute boot-related GPIO8/9.

Simulator terminal	Function	ESP32 destination	Connection requirement
1 or 2	Signal ground	GND	Confirm correct common ground
4	IAT	GPIO0	Through its own 2:1 divider after verification
5	ECT	GPIO1	Through its own 2:1 divider after verification
6	O2	GPIO2	Tested direct signal path; measured maximum approximately 1.25 V
7	MAP	GPIO3	Through its own 2:1 divider after verification
8	TPS	GPIO4	Through its own 2:1 divider after verification
3	RPM frequency	GPIO5	Leave disconnected until pulse interface is defined
9	Switch 9	GPIO6	Leave disconnected until voltage and identity are known
10	Switch 10	GPIO7	Leave disconnected until voltage and identity are known
11	3.3 V supply	No connection initially	USB powers ESP32
12	5 V supply	No connection initially	USB powers ESP32

The Arduino ADC attenuation setting changes measurement behavior. It does not make a GPIO tolerant of excessive voltage. Design the input for the ESP32-C6 ADC range of approximately 0-2.5 V at its maximum attenuation, rather than assuming the 3.3 V supply is the full ADC measurement range.

4 Voltage divider circuit

For the current simulator-only test, use two equal resistors per channel to halve the signal. Start with 10 kohm resistors, preferably 1% tolerance, rated 0.25 W. Four independent dividers require eight resistors. Do not remove or reuse resistors already installed on the simulator; they form part of its existing circuit.

```
Simulator signal terminal
          |
       R1 10 kohm
          |
          +---------- ESP32 analog GPIO
          |
       R2 10 kohm
          |
Simulator GND ------- ESP32 GND
```

R1 is between the signal terminal and midpoint. R2 is between the midpoint and ground. The GPIO connects to the midpoint. Do not simply place one resistor in series; that alone is not this divider.

The relationship is Vgpio = Vsignal × R2 / (R1 + R2). With equal resistors, Vgpio = Vsignal / 2 and reconstructed Vsignal = 2 × Vgpio.

Channel	Measured maximum before divider	Expected midpoint maximum
IAT	3.385 V	1.6925 V
ECT	3.575 V	1.7875 V
MAP	4.25 V	2.125 V
TPS	4.26 V	2.130 V

These are nominal calculations. Resistor tolerances, source impedance and ADC accuracy affect actual readings. The divider loads the simulator with approximately 20 kohm. Check the simulator voltage before and after attaching it. If it changes materially, obtain the output circuit information and use suitable buffering instead of assuming a factor of two is accurate.

This divider scales the measured range; it is not isolation or protection against an unexpected 12 V fault. A 12 V input would nominally create 6 V at its midpoint. Production integration needs fault protection matched to the actual circuit. Adding a real ECU later requires reevaluating input loading and grounding.

5 Parts and equipment

- Eight 10 kohm resistors, 1% tolerance preferred, 0.25 W, for four independent dividers.
- Breadboard or suitable soldered interface board and insulated hookup wires.
- Multimeter suitable for the voltage range being measured. A meter that reliably measures below 0.8 V is needed for low endpoints.
- Oscilloscope or other suitable pulse-measurement equipment for RPM amplitude and waveform verification.
- USB cable for the ESP32. The current testbench supply remains separate.

Do not choose the RPM or switch interface from these analog resistor values alone. Their electrical characteristics are still unknown.

6 Assembly and commissioning procedure

1. Switch off testbench power and unplug ESP32 USB. Keep all unverified signal wires disconnected.
2. Build one divider first, preferably MAP. With the GPIO disconnected, check its connections and resistor values. Measure resistance only with power off.
3. Power the testbench alone. Measure terminal 7 to simulator GND at several knob positions, including maximum, with and without the divider attached. Record any loading effect.
4. Measure the divider midpoint to GND. At the observed 4.25 V maximum, expect approximately 2.125 V. Confirm the midpoint remains within the intended ADC range through the full knob movement. The HT123 can check these high-end values, but low readings may remain unverified.
5. Switch off testbench power. Connect the verified midpoint to GPIO3 and connect common ground. Keep the unverified channels disconnected.
6. Power the ESP32 through USB, then the testbench. Read MAP pin voltage and compare with the midpoint measurement. Record both. Do not enable or interpret disconnected inputs as real measurements.
7. Only after the first divider behaves correctly, repeat the procedure independently for IAT to GPIO0, ECT to GPIO1 and TPS to GPIO4.
8. Use a factor of two only for channels actually fitted with verified equal-resistor dividers. O2's tested direct path uses a factor of one.
9. Verify signal tracking at multiple knob positions and save results. A maximum-only comparison is insufficient for full calibration.

Stop if wiring does not match the diagram, the midpoint exceeds the intended ADC range, the simulator output shifts materially, readings clip, or the board behaves abnormally. Investigate the interface before continuing.

7 Arduino voltage acquisition

The following helper demonstrates voltage reconstruction after the interfaces have been verified. It is a code example, not a complete sketch and not a claim that the existing dashboard already applies these scale factors.

```
#include <Arduino.h>

float readVoltage(uint8_t pin, float scale) {
  constexpr int samples = 16;
  uint32_t total = 0;
  for (int i = 0; i < samples; i++) {
    total += analogReadMilliVolts(pin);
    delay(2);
  }
  return total / (1000.0f * samples) * scale;
}

// Configure only verified and connected channels in setup():
// pinMode(pin, INPUT);
// analogReadResolution(12);
// analogSetPinAttenuation(pin, ADC_11db);

// After verification:
// float mapSignalV = readVoltage(3, 2.0f); // equal divider fitted
// float o2SignalV  = readVoltage(2, 1.0f); // verified direct O2 path
```

The firmware should preserve both the voltage measured at the GPIO and the reconstructed simulator voltage. Label the scale factor and calibration status. Do not multiply all channels indiscriminately.

For native USB serial, use USB CDC On Boot Enabled and the native USB connector. For the COM or CH343 USB-to-UART connector, use USB CDC On Boot Disabled for this sketch's Serial output. Re-upload after changing the setting and use a 115200 baud monitor. The standalone O2 sketch replaces the dashboard firmware when uploaded.

8 Dashboard and calibration behavior

The prepared Live monitor firmware displays millivolts at the ESP32 inputs, frequency and raw switch levels. It does not currently reconstruct divider-scaled simulator voltages or convert measured inputs to calibrated temperature, pressure or RPM. If it is uploaded, acquisition starts disabled. Disconnected analog and digital inputs may float; their values do not prove a connection.

Once the dividers are verified, update the dashboard to show both ADC pin voltage and reconstructed signal voltage. Use measured data labels and keep simulated sessions separate. Leave channels unavailable when no verified signal source is connected.

Temperature conversion requires the correct sensor curve or simulator calibration, especially for ECT and IAT. MAP needs confirmed calibration; TPS percent needs confirmed endpoints. The O2 voltage must not be converted to an accurate AFR with a universal linear formula. An actual AFR source has not been established.

RPM = frequency × 60 / pulses per revolution only applies when that pulse relationship is appropriate. Obtain the client's mapping before displaying RPM. A multimeter's average voltage does not establish the pulse's high voltage. One switch is reported to be brake, but which terminal and active state remain unknown.

These readings monitor simulator inputs. They do not certify ECU injection, ignition, ABS hydraulics, or traction-control operation, particularly because no ECU is connected in the present setup.

9 Measurement record

Complete one row for each tested knob position. Save the actual resistor values or interface ratio separately.

Channel and knob position	Terminal voltage	GPIO midpoint voltage	ESP32 reading	Reconstructed voltage	Notes
MAP maximum	4.25 V reported	To measure	To measure	To calculate	Check source loading
MAP middle	To measure	To measure	To measure	To calculate	
IAT maximum voltage	3.385 V reported	To measure	To measure	To calculate	
ECT maximum voltage	3.575 V reported	To measure	To measure	To calculate	Low endpoint unverified
TPS maximum voltage	4.26 V reported	To measure	To measure	To calculate	Low endpoint unverified
O2 maximum	1.248 V reported	Direct path	1.255 V reported	1.255 V	Basic acquisition verified

10 What this testbench is intended to do

The client wants an existing motorcycle testbench upgraded with a web interface. The ESP32 should make the bench easier to observe, record and explain. It should initially monitor the electrical signals produced by the physical knobs. It does not need to replace the simulator's circuits or physically move the throttle.

Based on the supplied photos and signal list, the bench probably imitates motorcycle sensor inputs so a technician can investigate a compatible engine ECU or ABS module without using a complete motorcycle. This is an interpretation, not a confirmed specification for every function. In the present setup, no ECU is attached: we are validating the simulator-to-ESP32 measurement path.

A module test needs a chain of evidence: a known test condition, the signal supplied to the module, the module's recognition of it, and the resulting output. A dashboard that only shows the knob voltage establishes the input condition. Injector commands, ignition commands, diagnostics or ABS pump/valve behavior would require separate observation interfaces.

```
CURRENT SETUP
12 V supply -> physical simulator knobs -> sensor signal outputs
                                          |
                                  scaling where required
                                          |
USB supply -> ESP32-C6 ADC inputs -> Wi-Fi dashboard / recorded data

FUTURE ECU TEST
Simulator signals -> compatible ECU harness -> ECU under test
                          |                        |
                    input monitoring      output / diagnostic interface
                          |                        |
                          +------ dashboard ------+
```

There are three distinct operating purposes. Demo generates illustrative data for presentation. Monitor acquires physical inputs. A controlled test would require supported hardware commands and actual response evidence. The current ESP32 inputs do not establish a control interface to the simulator or an ECU.

11 Client requirements and differences from actual measurements

The original ECU TESTBENCH DISPLAY document specifies proposed display conversions. Those values must not be silently substituted for what the board actually produces.

Signal	Original document input	Original display target	Actual evidence and implication
ECT	3.0 to 0.43 V	20 to 100 °C	Measured high end 3.575 V; low endpoint unverified. Confirm calibration before displaying temperature.
IAT	3.0 to 1.72 V	30 to 50 °C	Measured 1.660-3.385 V. Actual endpoints differ from the proposed document values.
MAP	0.78 to 4.0 V	20 to 101.3 kPa	Measured maximum 4.25 V. Pressure corresponding to this endpoint is unknown.
TPS	0.6 to 3.5 V	Voltage display	Measured maximum 4.26 V. Low reading zero is unverified.
O2	0 to 1.2 V	Voltage and lambda symbol	Measured maximum around 1.25 V. Voltage acquisition works; lambda calibration is not established.
AFR	0.7, 1.0, 1.2 V	10.3, 14.7, 17.6	These numbers resemble lambda × 14.7. The meaning of the input requires clarification.
RPM	0 to 300 Hz	0 to 15000 rpm	Client identifies terminal 3 as RPM, but its conversion is unconfirmed.
Vehicle speed	0 to 1500 Hz	0 to 250 km/h	No separate vehicle-speed source is identified on the current connector.

Signal	Original document input	Original display target	Actual evidence and implication
Front and rear wheel speed	Each 0 to 1500 Hz	Up to 250 km/h	The current RPM wire does not establish either wheel-speed channel.
Battery and two reference readings	Inputs unspecified	0 to 30 V display scales	No confirmed measurement terminals for these dashboard readings.
Smart key and winker	On/off	Indicators	No confirmed assignments to terminals 9 or 10.
ABS and TCS	On/off	Indicators	No confirmed real module status source on the present connector.

The handwritten sketch also differs from the document in some ranges: speed appears to reach 300 instead of 250, IAT appears to begin at 20 instead of 30 °C, and some voltage/frequency ranges differ. The sketch includes current and duty-cycle fields without identifying a measurement source. Treat these as items to resolve with the client, not additional verified channels.

The terminal drawing appears to mark a frequency range around 0-1500 Hz at terminal 3, whereas the earlier document uses 0-300 Hz for RPM. The client has clarified the function as RPM. Confirm the actual frequency range and calibration instead of assuming either drawing is authoritative for conversion.

12 Engine sensor meanings and testing context

ECT engine coolant temperature

ECT tells the ECU how hot the engine coolant is. It can influence starting fuel, injection timing and ignition strategy. A common sensor uses an NTC thermistor: resistance falls as temperature rises, and its ECU measurement circuit often produces a falling voltage. On a bench, a cold-to-warm setting tests whether the module recognizes temperature changes and responds appropriately. Source: https://www.hella.com/techworld/us/technical/sensors-and-actuators/test-coolant-temperature-sensor/

For our simulator, the direction reported by the operator is consistent with voltage falling toward the high-temperature end, but the low endpoint remains unmeasured reliably. The knob does not heat coolant; it represents a temperature electrically. A normal-looking voltage cannot prove a physical cooling system is working. The original 20-100 °C target is a requested display range, not a measured temperature.

Do not linearly label 3.575 V as 20 °C solely because it is the measured maximum. Obtain the intended calibration curve. A lookup table is suitable when the simulator or ECU uses nonlinear temperature conversion. Record whether the voltage represents a standalone simulator output or a sensor circuit connected to an ECU pull-up.

IAT intake air temperature

IAT describes incoming air temperature. Together with pressure, it helps estimate intake air mass for fueling. At a given pressure, colder air is denser. Temperature and pressure sensing are used to support fuel quantity and air management. Source: https://www.bosch-mobility.com/en/solutions/sensors/intake-manifold-and-boost-pressure-sensor/

The simulator's IAT signal was reported between 1.660 and 3.385 V. Calling the lower-voltage setting the maximum may refer to maximum temperature, not maximum voltage. Record voltage and intended temperature in separate columns. The direct connection is not suitable for measuring the full range with this ADC configuration; its divider reduces the reported range to approximately 0.830-1.693 V.

IAT is separate from ECT. A warm engine can draw cool intake air, and a stationary engine can sit in hot air. The demo should not force both temperatures to follow vehicle speed. A guided test should vary IAT while holding unrelated simulator inputs steady.

MAP manifold absolute pressure

MAP indicates absolute pressure in the intake manifold and is used in engine-load estimation. Absolute pressure is different from gauge pressure or a vacuum magnitude. Approximately 101.3 kPa is a standard atmospheric reference, not a universal local barometric reading.

In a naturally aspirated engine, pressure with the engine stopped is generally near ambient. Closed-throttle running creates lower manifold pressure; opening the throttle under load generally raises it. These are contextual examples, not a calibration curve for this testbench. On the bench, the knob represents pressure electrically; it does not produce intake vacuum.

The measured maximum is 4.25 V, above the original proposed 4.0 V endpoint. Do not extrapolate a pressure beyond 101.3 kPa without a confirmed transfer function. Keep the dashboard on measured voltage until the client supplies pressure-versus-voltage calibration. After a verified equal divider, show approximately 2.125 V at GPIO3 and reconstruct approximately 4.25 V at the simulator output.

TPS throttle position sensor

TPS measures throttle valve angle and contributes to engine air-filling prediction. It is distinct from engine RPM and, on electronic throttle systems, can be distinct from accelerator grip position. Source: https://www.bosch-mobility.com/en/solutions/control-units/engine-management-system-2w/

The present TPS knob generates a voltage; changing it does not physically open a motorcycle throttle. The measured high end is 4.26 V, rather than the proposed 3.5 V. Verify the true low endpoint before displaying percent. A potential normalized display uses (signal voltage minus verified closed value) divided by (verified open value minus verified closed value), multiplied by 100. Apply it only once those meanings are confirmed. Clamping a display to 0-100% must not conceal an out-of-range electrical condition.

O2 exhaust oxygen feedback

An actual exhaust oxygen sensor provides information used for mixture feedback. A common narrowband zirconia sensor responds sharply around the stoichiometric point: lower voltages typically represent lean operation and higher voltages rich operation, once operating conditions are suitable. That behavior is not a general linear AFR scale. Source: https://ngksparkplugs.com/wp-content/uploads/2019/04/Oxygen-Sensors.pdf

Our bench produces an imitation O2 voltage. No exhaust gas is being measured in the present setup. We verified that GPIO2 follows the physical O2 knob and that its high-end reading of 1.255 V closely agrees with the meter's 1.248 V. This is the strongest current acquisition evidence. It does not identify the sensor type accepted by a particular ECU or prove that 1.25 V is a normal value for that sensor.

Continue displaying measured voltage. A useful comparison records low, middle and high settings with a suitable meter. Do not convert this signal into measured AFR or measured lambda unless the simulator provides a documented encoding.

AFR and lambda

AFR is the mass of air divided by the mass of fuel. For conventional gasoline, an approximate stoichiometric reference is 14.7:1. Lambda is actual AFR divided by the fuel's stoichiometric AFR; lambda 1 is stoichiometric, below 1 rich, above 1 lean. Fuel composition changes the stoichiometric reference. Source: https://www.ngkntk.com/fileadmin/middle_east/downloads/lambda_catalogue_2017-18.pdf

The supplied AFR points are revealing: 0.7 × 14.7 = 10.29, 1.0 × 14.7 = 14.7, and 1.2 × 14.7 = 17.64. They resemble lambda conversion. This is a numerical observation, not proof that the listed volts actually encode lambda. Ask whether the input numbers should be lambda without units, or whether a separate controller generates a documented analog encoding. The connector has no separate AFR terminal identified.

The demo uses an independent simulated lambda setting for its illustrative AFR. Actual O2 voltage is not used to calculate that demo AFR. In a live session, mark AFR unavailable until its actual source exists.

13 RPM wheel speed and switching signals

RPM frequency

Terminal 3 is confirmed by the client as an RPM-related frequency output. Frequency is cycles per second in Hz; RPM is shaft revolutions per minute. The conversion depends on pulse meaning. One pulse per revolution gives RPM = 60 × Hz; two pulses per revolution gives RPM = 30 × Hz. Thus 100 Hz can mean 6000 or 3000 RPM in those respective examples.

The original mapping of 300 Hz to 15000 RPM implies a factor of 50 RPM per Hz. That is a clue that this may be a custom display signal rather than a simple one-pulse-per-revolution source. Do not assign a pulse count by guessing. Actual ECU crank inputs may require position patterns and synchronization in addition to frequency. Source for crank speed and position use: https://www.bosch-mobility.com/en/solutions/control-units/engine-management-system-2w/

The prepared monitor counts rising edges on GPIO5 over an approximately one-second window. It reports Hz. At very low frequency, simple counting is coarse; precision may need period measurement later. Input voltage, negative excursions, output type and suitable conditioning must be confirmed before connecting this signal.

Vehicle and wheel speeds

The dashboard sketch contains vehicle speed and front/rear wheel speed. The currently identified connector provides one RPM frequency terminal, not three separate motion signals. Do not relabel RPM as road speed. Converting wheel frequency to speed requires wheel circumference and pulses per revolution: speed in km/h = 3.6 × circumference in metres × frequency in Hz / pulses per wheel revolution.

The original frequency-to-speed endpoints are proposed display settings. They do not establish a wheel geometry or a verified input. Keep these fields unavailable in live mode until sources are identified; keep existing illustrative values within Demo mode.

Switches 9 and 10

Pull-up and pull-down describe electrical arrangements, not the switch's purpose. The client reports one switch is brake, but has not identified which. Before use, measure each terminal relative to ground in both positions, identify whether it is a dry contact or powered output, and obtain the active state. A dry contact may require a pull resistor; the prepared code currently enables no internal pull-up or pull-down.

For now, raw digital readings mean HIGH or LOW only. They must not be labeled brake applied, smart key accepted, ABS active or TCS active by assumption. Unconnected inputs may fluctuate. Software cannot protect an input from a switch output at 5 or 12 V.

14 ABS and traction control

ABS uses wheel-speed information to detect impending locking during braking and modulates hydraulic brake pressure. Motorcycle versions may control one or both wheels. Source: https://www.bosch-mobility.com/en/solutions/driving-safety/antilock-braking-system-2w/

For a real test, front/rear speed emulation must match the target module's electrical interface and wheel calibration. Displaying two different numbers alone is not proof of detection or hydraulic response. Module diagnostics, pump/valve command measurements and hydraulic pressure testing are different levels of evidence. An ABS warning lamp also differs from an intervention indicator.

TCS addresses excessive driven-wheel slip during acceleration. Depending on the system, it can coordinate torque changes with engine control; advanced stability systems use additional motion sensing. Source: https://www.bosch-mobility.com/en/solutions/driving-safety/motorcycle-stability-control/

The current connector does not identify separate front/rear inputs, module communications, hydraulic output measurements or actual ABS/TCS status. Therefore the project's current live capability cannot demonstrate ABS or TCS operation. Those remain future integration requirements. Simulated lamp toggles and the demo's unequal-wheel-speed scenario are presentation functions.

15 Other display fields and source identification

Field	Intended usefulness	Present limitation
Battery voltage	Observe ECU or bench supply stability	No measurement input identified. The 12 V supply description is not a live voltage reading.
Reference voltage 1 and 2	Check defined sensor supply/reference circuits	Two measured reference sources are not identified. A 0-30 V display range is not a normal reference specification.
Battery current	Observe consumption under test	Requires a current sensing interface and a defined measurement point.
Duty cycle	Observe active time as a fraction of a signal period	Must identify which waveform or actuator it represents.
Smart key	Indicate authorization state	A toggle alone cannot establish a real immobilizer exchange.
Winker	Indicate turn-signal state	No measured source identified.
Check engine	Observe a module malfunction indication	No measured lamp output or diagnostic source identified.
ABS and TCS lamps	Present module warning/status	Source and meaning unconfirmed.

Terminals 11 and 12 are power supplies, not automatically separate voltage-sensing inputs. Do not wire those rails to ADC pins without a defined sensing interface. The ESP32 currently does not measure supply current.

16 Firmware and dashboard versions

Two firmware purposes were used during development. The web dashboard firmware provides a Wi-Fi access point and demo tools, with a later live monitor addition. The standalone O2 sketch provides serial voltage readings and replaces the web server when uploaded. Seeing O2 serial output does not establish that the board is simultaneously serving the dashboard. A previously loaded browser page can remain visible even if its server is no longer running.

The web access point is named ECU-Testbench-C6, with password ECUdemo2026 and local address http://192.168.4.1/. It is a local network without internet access. The captive portal may produce a sign-in prompt after connection; the prompt depends on the client device. This computer was reported to have Ethernet but no Wi-Fi adapter, so a Wi-Fi-capable phone or computer is needed to connect directly.

The web demo has independent sensor overrides, guided illustrative scenarios, signal history, event recording, CSV export and printable reports. Start Engine and Brake change simulation state; they do not start a real engine or actuate an ABS unit. Simulated records use explicit simulated labels and do not produce module health certification.

The prepared live monitor is input-only. It defaults to acquisition disabled, reads voltages at GPIO0-4, counts GPIO5 pulses and reads GPIO6/7 states when enabled. The interface scale factors have not been

implemented for reconstructed simulator voltage. Live CSV is separate from demo records, and disconnected readings must not be interpreted as valid channels. The present O2 serial test demonstrated only GPIO2 acquisition.

Before restoring the web firmware, set the serial output configuration to the chosen USB connector and keep the hardware signal interface verified. Do not enable all channels simply because O2 works. Request an O2-only live view or finish commissioning each additional channel before presenting it as measured.

17 Calibration and acceptance evidence

For every channel, retain three different quantities: source terminal voltage, GPIO voltage after the interface, and interpreted engineering value. Also retain timestamp, scale factor, calibration version and quality status. A usable display might say MAP source 4.25 V, ADC pin 2.125 V, pressure unavailable until calibrated.

For an equal divider, source reconstruction uses a factor of two. With unequal actual resistors, use (R1 + R2) / R2. If the source is loaded, the reconstructed value refers to the loaded terminal voltage; it does not recover the original unloaded voltage. Compare measurements under the same wiring and power conditions.

Acceptance should include multiple knob positions, repeatability, stable supply and correct channel identity. Choose numeric tolerances using the meter specification, resistor tolerances and ADC characteristics, rather than inventing a pass threshold from a single reading. Check whether the direction of change matches the intended sensor. Save discrepancies instead of hiding them with forced endpoint correction.

For temperature and pressure, obtain a table of terminal voltage versus intended engineering value. Use a documented transfer function or interpolation of verified points. For RPM, obtain frequency versus RPM examples from the client. For switches, obtain identity and active state. Every missing calibration should remain visible as a limitation.

18 Troubleshooting

Observation	Possible cause	Next check
Meter shows zero at a low setting	Signal below HT123's specified range or actual zero	Verify with a suitable low-voltage meter; do not assume zero
ESP32 voltage does not follow the knob	Wrong terminal, wrong GPIO, missing ground or incorrect interface	Power off and trace the wiring; compare terminal and midpoint readings
ADC reads near full scale	Signal beyond ADC range or scaling error	Disconnect and verify midpoint voltage; clipping is not a calibration method
Disconnected channel shows numbers	Floating input	Mark channel unavailable; do not treat it as sensor data
Reading is half the expected source voltage	Divider installed but display shows ADC voltage	Apply verified scale factor to a separately labeled source field
Simulator output changes when divider is attached	Output source loading	Verify output circuit; use appropriate buffering if necessary
No serial output	Connector, CDC setting, selected port or monitor conflict	Match native USB with CDC enabled or COM/CH343 with CDC disabled; re-upload
Dashboard no longer connects after serial test upload	Standalone sketch replaced web firmware	Restore the appropriate web firmware; a cached page is not live server evidence
Hotspot not visible	Board power, firmware or client Wi-Fi availability	Confirm board startup and use a Wi-Fi-capable device
Live endpoint unavailable	Monitor disabled, warming up, wrong firmware or connection lost	Check firmware and enabled state; never substitute demo values as live data

19 Next implementation sequence

1. Obtain the eight resistors and validate one divider with MAP, keeping its GPIO disconnected during midpoint measurement.
2. Commission IAT, ECT and TPS separately. Verify low endpoints with a suitable meter.
3. Restore the web dashboard with only verified inputs presented as measured. Add the four divider scale factors and keep O2 factor one.
4. Measure RPM and switch electrical characteristics and fit appropriate interfaces. Confirm calibration and identities.
5. Add calibrated temperature, pressure, TPS percent and RPM only after client confirmation.

6. If an actual ECU or ABS module is supplied, obtain its model-specific harness, supported communications and output test method. Reevaluate circuit loading and fault protection.
7. Build guided live tests around known inputs and actual response evidence. Keep demo examples and measured reports separate.

20 References

- Espressif ESP32-C6 datasheet: ADC characteristics, pin functions and electrical limits. https://www.espressif.com/sites/default/files/documentation/esp32-c6_datasheet_en.pdf
- Espressif Arduino ADC API: analogReadMilliVolts, resolution and attenuation settings. https://docs.espressif.com/projects/arduino-esp32/en/latest/api/adc.html
- Habotest HT123 manual: low-voltage measurement limitation. The manufacturer-hosted PDF was indexed during review, but direct retrieval failed; verify the specifications against the manual supplied with your meter. Do not treat zero readings at low endpoints as confirmed measurements.
- Project evidence: client terminal drawing, supplied ESP32-C6 board diagram, and user-reported measurements and O2 serial output during this session.
- Original client requirements: ECU TESTBENCH DISPLAY.docx and sample interface.jfif. Requirements and sketches are reference material; they do not prove measured hardware behavior.
- Manufacturer sources for sensor and ABS explanations are linked beside the corresponding sections. Their operating examples do not define this simulator's electrical calibration.